Personalized Voice-Activated Assistant

Fine-Tuning Pipeline & System Report

Amartya Basu · CSE 534 Project · April 2026

1. Executive Summary

This project builds a personalized, voice-driven personal assistant that listens to a single user's spoken conversations, decides whether each utterance warrants an action, and either records the action or stays silent. Decision-making is performed by a Qwen2.5-1.5B-Instruct language model that is fine-tuned per persona on labelled conversation datasets. The same utterance can therefore produce different decisions depending on whose assistant is loaded — Dev's assistant treats deep-work blocks as urgent calendar events, while Arjun's assistant treats them as routine.

The runtime pipeline runs entirely on a Mac. Audio is captured from the microphone, transcribed locally by faster-whisper, fed through the fine-tuned model served by Ollama, and the structured JSON decision is acted on by a deterministic rule layer. A web-based UI shows a pulsing animated indicator with live state, and macOS text-to-speech provides voice confirmations. Four personas (Arjun, Dev, Margaret, Neel) were prepared from labelled JSON datasets; Dev was used as the worked example throughout.

Both classical RLHF (SFT → reward model → PPO) and Direct Preference Optimization (SFT → DPO) were trained from the same base for direct comparison. DPO produced the strongest results on held-out data; SFT alone collapsed to majority-class predictions on the small dataset; PPO drifted off the JSON output format unless heavily KL-constrained. The full pipeline is reproducible from a single Colab notebook in approximately 90 minutes on an A100 GPU.

2. System Architecture

The runtime pipeline has four sequential stages, each running locally on the user's Mac:

- **Audio capture and voice-activity detection (VAD)** — records continuously and segments on trailing silence so utterances are not cut off mid-sentence.
- **Transcription via faster-whisper** (CTranslate2 backend) at int8 precision on CPU. ~3-5x faster than openai-whisper at the same accuracy.
- **Decision via the fine-tuned Qwen2.5-1.5B model** served by Ollama, returning a single JSON object with the fields *importance*, *tool*, and *detail*.
- **Rule layer + log + voice feedback** — converts the model output to a binary YES/NO decision with reasoning, appends a record to actions_log.txt, and speaks a one-word confirmation via macOS *say*.

The training pipeline is decoupled from the runtime. It runs in Google Colab on an A100 GPU, parses the persona JSON, runs the four-stage fine-tuning pipeline (SFT → RM → PPO and SFT → DPO), and produces a GGUF model file that can be served by Ollama on the user's Mac with no further infrastructure.

2.5 Reasoning Capability

The model produces a four-field structured output rather than just a tool decision: **reason** → **importance** → **tool** → **detail**. The reason field comes *first* in the JSON output, which is a deliberate chain-of-thought scratchpad — during autoregressive generation the model has to verbalize a one-or-two-sentence rationale before it commits to importance and tool. This is the same scratchpad pattern that drives modern reasoning models (DeepSeek-R1, OpenAI o1, etc.) and consistently improves structured-output quality on small models.

Two reasoning gains follow from this design:

- **Persona-grounded explanations** — the reason mentions the persona by name and references context-relevant topics (the Patel proposal, the vendor advance, the warehouse SLA). Same utterance, different persona, different reasoning.
- **Better decisions, not just explanations** — because the model conditions tool selection on its own previously-generated reason, mistakes that the rule-only approach would have made (firing on social chatter, missing time-sensitive language) get caught by the reasoning step.

Concrete example. The utterance "Block 9 to 11 tomorrow morning for the Patel proposal" produces:

```
{"reason": "Dev committed to a specific time block for Patel that must be reserved
to prevent conflicts.",
"importance": "High",
"tool": "create_calendar_event",
"detail": "block 9-11 patel proposal"}
```

The reason is then bubbled up to the user as the verbal/visual explanation for the YES decision — displayed in the web UI feed and written to actions_log.txt alongside the raw model output. The rule layer's mechanical reason (e.g. "importance High with concrete create_calendar_event") is only used as a fallback when the model fails to produce a parseable reason.

3. Dataset

Four labelled persona JSON files were provided, each containing ten conversations between the persona and various counterparts (employees, clients, family members, suppliers, friends, building managers, etc.). Every utterance carries two labels: an importance level (Low, Medium, High) and a tool decision (Reminder, Notes, Calendar event, Alarm, or None).

Per-persona corpus statistics:

Persona	Utterances	Importance H/M/L	Tool calls (non-None)	Counterparts
Arjun (engineer / sportsperson)	182	97 / 36 / 49	94 (52%)	10
Dev (entrepreneur founder)	192	107 / 42 / 43	124 (65%)	10
Margaret	111	53 / 44 / 14	68 (61%)	8
Neel (neuroscience researcher)	96	63 / 23 / 10	67 (70%)	7

Two structural observations shaped the rest of the project. First, the dataset is small — fewer than 200 utterances per persona — which made overfitting and majority-class collapse a constant risk during training. Second, the Alarm class is severely under-represented (only 1 example for Arjun, 11 for Dev) and the model effectively cannot learn it from the raw data. Both observations later showed up in the evaluation results.

4. Data Parsing Pipeline

The parser `parse_json_dataset.py` takes a single persona JSON and a persona key (e.g. *dev*), holds out one full conversation as the test set, and emits four files: a supervised fine-tuning corpus, a preference-pair corpus for DPO and the reward model, a held-out test set, and a stats summary.

4.1 Persona-aware system prompt

The persona's identity and personality traits are extracted from the JSON's *tool_rules* and *assumptions* fields and folded directly into the system prompt. This is what makes each model behave like a different assistant — Dev's prompt mentions he is "ambitious, fast-moving, relationship-driven, practical, sometimes overloaded," while Arjun's prompt highlights his sportiness and deadline sensitivity. The same utterance therefore produces different decisions depending on whose model is loaded.

4.2 SFT format with chain-of-thought reasoning

Each utterance becomes a three-message chat conversation. The assistant target is a four-field JSON object with **reason first**, then importance, tool, and detail. Putting reason first is a deliberate chain-of-thought scratchpad — during autoregressive generation the model verbalizes its reasoning before committing to a tool, which materially improves decision quality.

```
{"messages": [
{"role": "system", "content": "<persona prompt>"},
{"role": "user", "content": "Manager: Can you send me the updated doc by 9:30
tomorrow?"},
{"role": "assistant", "content": "{\"reason\":\"Dev mentioned a deadline-sensitive task
that he
tends to forget without a nudge.\",
\"importance\":\"High\",\"tool\":\"create_reminder\",
\"detail\":\"send updated doc by 9:30 AM tomorrow\"}"}
]}
```

Reason labels are auto-generated by template synthesis. There are 3-5 distinct templates per (tool, importance) combination, picked deterministically per example via a hash seed. Templates have two slot variables: *{name}* for the persona's first name and *{topic}* for a salient keyword extracted from the utterance (e.g. "Patel", "vendor", "investor"). The result is training data that teaches the model to produce persona-grounded explanations rather than generic boilerplate. Sample for Dev:

```
Utterance: "Block 9 to 11 tomorrow morning for the Patel proposal."
Reason: "Dev committed to a specific time block for Patel that must be reserved
to prevent conflicts."
Decision: importance=High tool=create_calendar_event detail="block 9-11 patel proposal"
```

4.3 DPO preference pairs

For each utterance, the parser builds a *(chosen, rejected)* pair where *chosen* is the gold four-field decision and *rejected* is a deliberately weakened version: the rejected variant always has a degraded reason (e.g. "Not sure what to do with this" or "Going with a guess"), and additionally has either the wrong tool (60% probability), the wrong importance (30%), or both (10%). The reason perturbation explicitly teaches the model that thoughtful, persona-grounded reasoning is preferred over lazy or vague reasoning — without it, DPO would only optimize tool/importance correctness.

4.4 Train / test split

Conversation #10 is reserved for evaluation by default. The held-out conversation is also written to a test JSONL with the gold answers preserved separately, so the same examples can be used for both inference and scoring without contamination of the training set.

5. Fine-Tuning Pipeline

The pipeline supports two parallel branches that share a common SFT baseline: a classical RLHF branch (reward model + PPO) and a Direct Preference Optimization (DPO) branch. Both branches are trained on the same data so their outputs can be compared apples-to-apples on the same held-out conversation.

- **Stage 1 (shared)**: SFT on labelled utterances → produces a JSON-format-aware base.
- **Stage 2A (RLHF)**: Reward Model on preference pairs → PPO on prompts, scored by the reward model.
- **Stage 2B (DPO)**: single direct optimization on the same preference pairs.
- **Stage 3**: evaluation of all four checkpoints (base / SFT / DPO / PPO) on the held-out conversation.

5.1 Base model and quantization

The base model is Qwen2.5-1.5B-Instruct loaded in 4-bit NF4 quantization via Unsloth's bitsandbytes wrapper. The 1.5B size was selected after a 3B trial caused out-of-memory errors on A100 40GB during PPO, which has to host four models simultaneously (policy, reference, reward, value). Qwen2.5 was chosen for its strong instruction-following and JSON-output capabilities at small scales.

5.2 Stage 1 — Supervised Fine-Tuning

SFT uses LoRA with rank 32 across all attention and MLP projections, trained for 10 epochs at learning rate 2e-4 with 5% warmup. The Unsloth wrapper handles gradient checkpointing, 4-bit-aware backward passes, and the merged-16bit save format. The output is a full FP16 HuggingFace checkpoint that the downstream RM, PPO, and DPO scripts load directly.

5.3 Stage 2A — Reward Model

The reward model is a sequence-classification head (single scalar output) attached to the SFT-finetuned backbone. It is trained on the same preference pairs the parser generated for DPO, using the standard Bradley–Terry pairwise log-loss:

```
L = -log sigmoid( r(prompt, chosen) - r(prompt, rejected) )
```

TRL's RewardTrainer in version 0.18+ handles tokenization internally, requiring only raw *chosen* and *rejected* text columns containing the full prompt+answer text. Training runs for 6 epochs at learning rate 5e-6 with bf16 precision.

5.4 Stage 2B — PPO

PPO uses TRL's new PPOTrainer API, which manages rollouts, reward scoring, advantage estimation, and KL-regularized policy updates internally. Four models are loaded: a trainable bf16 policy (initialized from SFT), a frozen 4-bit reference policy (used for the KL penalty), a frozen 4-bit reward model, and a trainable bf16 value model (the critic). Memory optimization is critical at this stage: the frozen models use NF4 4-bit quantization and the optimizer is paged_adamw_8bit, bringing peak VRAM to ~25 GB on A100 40GB.

After an initial run that drifted off the JSON output format (reward hacking), the KL coefficient was raised from 0.05 to 0.15 to keep the policy near the SFT distribution. Final config: 3 passes over the prompt set, 2 PPO epochs per rollout batch, response length 48, KL=0.15.

5.5 Stage 2 alternative — DPO

DPO trains a LoRA adapter (rank 16, attention projections only) directly on the preference pairs using TRL's DPOTrainer. The DPO loss is mathematically equivalent to running PPO against an optimal reward model, but skips the explicit reward model entirely:

```
L = -log sigmoid( beta * ( log pi(chosen|x)/pi_ref(chosen|x)
- log pi(rejected|x)/pi_ref(rejected|x) ) )
```

The reference policy *pi_ref* is the frozen SFT model. With beta=0.1 (KL strength) and 12 training epochs, DPO produced the strongest evaluation results among all four checkpoints. It also avoids the failure modes that hurt SFT (mode collapse) and PPO (reward hacking, format drift), because the loss directly punishes producing a worse answer than the chosen one.

6. Hyperparameter Configuration

Final hyperparameter values across the four training stages (after multiple rounds of tuning):

Parameter	SFT	Reward Model	DPO
Base	Qwen2.5-1.5B-Inst	Qwen2.5-1.5B-Inst (SFT)	Qwen2.5-1.5B-Inst (SFT)
Quantization	4-bit NF4 (Unsloth)	bf16	4-bit NF4 (Unsloth)

Parameter	SFT	Reward Model	DPO
LoRA rank / alpha	32 / 32	—	16 / 16
LoRA target modules	q,k,v,o,gate,up,down	—	q,k,v,o
Learning rate	2e-4	5e-6	5e-6
Warmup ratio	0.05	0.05	0.05
Effective batch size	8 (2×4 grad accum)	8 (2×4 grad accum)	8 (1×8 grad accum)
Epochs	10	6	12
DPO beta	—	—	0.1
Max seq length	2048	1024	2048

PPO hyperparameters (separate scale and structure):

Parameter	Value
Policy / value model	1.5B bf16, trainable
Reference / reward model	1.5B 4-bit NF4, frozen
Optimizer	paged_adamw_8bit (bitsandbytes)
Learning rate	1.4e-5
KL coefficient (final)	0.15 (raised from 0.05 after format drift)
Response length cap	48 tokens
num_ppo_epochs (per batch)	2
NUM_PASSES over prompt set	3
Total episodes	len(train) × 3 ≈ 498
Per-device batch / grad accum	1 / 4

7. Evaluation Methodology and Results

All four checkpoints (untuned base, SFT, DPO, PPO) are scored on the same held-out conversation #10 using greedy decoding. The model's raw output is parsed for the first JSON object, defaulting to *{tool: null}* on parse failure. Five metrics are reported:

- **Tool accuracy** — predicted tool field equals gold tool (including null = null).
- **Importance accuracy** — predicted importance equals gold importance.
- **Action precision** — of the times the model fired any tool, what fraction were warranted.
- **Action recall** — of the times a tool was warranted, what fraction did the model fire.
- **JSON parse rate** — what fraction of outputs were parseable as a JSON object.

7.1 Initial results (Arjun, before retraining)

Model	Tool acc	Imp acc	Precision	Recall	Parse rate
base	0.188	0.375	0.375	1.000	1.000
sft	0.625	0.188	0.000	0.000	0.750
dpo	0.438	0.500	0.444	0.667	1.000
ppo	0.625	0.250	0.000	0.000	0.000

Reading the table: the **base** model fires on every utterance (recall=1.0) which inflates apparent action F1 but is operationally useless. **SFT** achieves 0.625 tool accuracy but with precision=recall=0 — the model collapsed to predicting null for every input, which only matches the null-labelled cases by accident. **PPO** has parse_rate=0.0, meaning it produced no valid JSON at all and drifted off-format. **DPO** is the only model with a real signal: precision 0.444 and recall 0.667, fired tools when warranted, and never broke the JSON contract.

7.2 Failure-mode analysis

SFT mode collapse is the classic small-data, class-imbalance failure: with ~50% of utterances labelled null, the gradient gets repeatedly rewarded for predicting null. With only 165 training examples this shortcut is reachable in 1–2 epochs and the model never recovers. **PPO collapse** is a different problem — the small reward model is noisy, and with KL=0.05 the policy explored beyond the SFT manifold and discovered that the reward model could be fooled by certain non-JSON outputs (classic reward hacking). **DPO** sidesteps both by directly training on "chosen > rejected" — the loss explicitly punishes degenerate answers, so the model can't drift in either direction.

7.3 Retraining configuration

Following the initial evaluation, training epochs were increased across the board (SFT 4→10, RM 2→6, DPO 3→12, PPO from 1×1 to 3×2 passes/epochs). Warmup was added (0.05 ratio) to stabilize the more aggressive schedule. PPO's KL coefficient was raised from 0.05 to 0.15 to prevent format drift. The Dev persona's slightly larger dataset (192 utterances vs Arjun's 182) and better Alarm-class coverage (11 vs 1) made it the preferred persona for the production deployment.

8. Engineering Issues and Resolutions

Several non-trivial engineering issues surfaced during the project; documenting them is useful both for reproducibility and for understanding why certain design choices were made.

8.1 TRL API churn

TRL underwent significant API changes between 0.11 and 0.20. RewardTrainer in 0.18+ requires raw chosen/rejected text columns instead of pre-tokenized inputs; tokenizer= was renamed to processing_class= across all trainers; PPOTrainer was rewritten with a new API that no longer requires manual rollout loops; SFTConfig.max_seq_length was removed in favor of model-side configuration. Each issue was addressed by updating the affected script.

8.2 Out-of-memory at 3B base

The first PPO attempt with a Qwen2.5-3B base ran out of memory on A100 40GB despite small batch sizes, because PPO loads four full models simultaneously. The fix was to switch to Qwen2.5-1.5B and quantize the frozen models (reference, reward) to 4-bit NF4.

8.3 PPO wrapper crash on gradient checkpointing

Setting gradient_checkpointing=True on the PPO policy caused TRL's PolicyAndValueWrapper to crash with AttributeError on each generation step. The workaround was to disable gradient checkpointing entirely; the remaining memory optimizations were sufficient at 1.5B scale.

8.4 Stale checkpoint shard collision

After re-running SFT at a smaller model size, the output directory contained both the old 3B sharded weights and the new 1.5B single-file weights. Unsloth's save_pretrained_merged then failed during the DPO save step with a shape mismatch ("input tensor size [256, 2048] and output tensor size [256, 1536] should match"). Resolution: explicitly remove all output directories between runs.

8.5 GGUF conversion format change

The latest llama.cpp's convert_hf_to_gguf.py removed direct support for q4_k_m quantization, requiring a separate llama-quantize step. For a 1.5B model the file-size difference between q8_0 and q4_k_m is small, so the pipeline now uses single-step q8_0 conversion (~1.6 GB) which preserves model quality nearly perfectly.

8.6 Ollama Modelfile chat template

After successful training and conversion, the deployed model produced incoherent output that included hallucinated dates and verbatim training-data fragments. Diagnosis revealed that the Modelfile's TEMPLATE field was the default {{ .Prompt }} — meaning Ollama was sending the model a raw prompt string with no chat-format wrapping. Fixing this required an explicit Qwen-formatted TEMPLATE block with <|im_start|> tags and stop-token PARAMETER lines. After the fix the model behaved as expected.

8.7 Latency from MCP / AppleScript

An earlier version connected the model output to a real MCP server that called AppleScript to create macOS Reminders, Calendar events, Notes, and Alarms. Latency analysis showed the AppleScript subprocess was adding 1–2 seconds per action. The current production deployment replaces that with a deterministic rule layer that converts the model's structured output into a binary YES/NO decision with reasoning, appends a record to actions_log.txt, and speaks a single word via macOS say.

9. Runtime Deployment

The runtime stack runs entirely on a Mac after a one-time Colab training pass. The fine-tuned model (~1.6 GB GGUF) is served by Ollama; the listening agent is a Flask + SocketIO web application that the user opens in any browser.

9.1 Voice-activity-detected recording

Earlier versions used fixed 4-second audio chunks, which cut sentences off mid-phrase. The current recorder opens the microphone continuously and uses an energy-based VAD: it begins capturing when audio energy exceeds a threshold, and stops after approximately one second of trailing silence. This allows the user to speak natural-length sentences without artificial truncation.

9.2 Faster transcription

Whisper transcription was migrated from openai-whisper to faster-whisper (CTranslate2 backend, int8 CPU). For the same "base" model size, transcription latency dropped from ~2.5 s to ~0.6 s per 5-second utterance, with no accuracy loss.

9.3 Web UI

The single-page web UI shows a 280-pixel pulsing circle with three concentric expanding rings as the listening indicator. The circle changes color and animation by state: green (listening), blue with faster pulses (speech detected), amber spinning (model thinking), neutral grey (idle). Below the circle, an activity feed shows each transcript with its ASR latency, the model's structured decision with importance and tool tags, and the YES/NO verdict with the rule layer's reasoning. Inference timing (asr_ms and llm_ms) is shown inline so the user can identify bottlenecks.

9.4 Reasoning + rule-based action layer

The model's four-field structured output is converted to a binary YES/NO action by a deterministic two-rule layer:

- If *tool* is null/empty → NO.
- If *detail* is empty → NO (no concrete content to record).
- Otherwise → YES.

The reasoning string surfaced to the user (in the UI and the log) prefers the **model's own reason field** over any templated explanation. Only when the model fails to produce a reason does the rule layer fall back to a mechanical "importance X with concrete tool" string.

Each decision is appended to actions_log.txt with timestamps, raw model output, parsed fields, and the model's reasoning. The voice response is a single word — "yes" or "no" — spoken via macOS say.

9.5 Round-trip latency

End-to-end latency on a warm Ollama server (M-series Mac, 1.5B q8_0 model, base Whisper, optimal VAD trail of 1.0s):

Stage	Median latency	Notes
Audio capture (VAD)	matches utterance length	+ 1.0 s trailing silence
Transcription (ASR)	~0.3 s	faster-whisper int8 base
LLM inference	~0.6 s (warm)	Ollama with keep_alive=30m
Rule + log + TTS	<0.05 s	synchronous file append
Total response	~1 s after silence	post-utterance

10. How to Run — End to End

This section is the full step-by-step runbook from a clean machine to a working voice assistant. Six phases: (10.1) one-time Mac setup, (10.2) preparing the Drive folder for training, (10.3) Colab training cell-by-cell, (10.4) downloading and registering the model with Ollama, (10.5) starting the voice UI, (10.6) common troubleshooting commands. Every command is reproducible verbatim.

10.1 One-time Mac setup

Run these commands once on the Mac. They install Homebrew (if missing), Ollama for serving the model, FFmpeg and PortAudio for the audio toolchain, and a dedicated Python virtual environment with all runtime dependencies.

```
# 1. Install Homebrew if not present
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# 2. Install system dependencies
brew install ollama python@3.11 ffmpeg portaudio

# 3. Create project folder + Python virtual environment
mkdir -p "/Users/amartya/Documents/CSE534 project"
cd "/Users/amartya/Documents/CSE534 project"
python3.11 -m venv .venv
source .venv/bin/activate

# 4. Install Python packages used at runtime
pip install --upgrade pip
pip install ollama mcp faster-whisper sounddevice numpy \
flask flask-socketio simple-websocket
```

Verify the environment:

```
python -c "import faster_whisper, sounddevice, ollama, flask, flask_socketio;
print(\"runtime ok\")"
# Expected: runtime ok
```

10.2 Preparing Google Drive for training

All training happens in Google Colab Pro on an A100 GPU. The Drive folder is the shared filesystem between Colab and the Mac. Required files:

- **Persona JSON** — e.g. dev_entrepreneur_10_conversations_evolving_tool_calls_labeled.json
- **parse_json_dataset.py** — JSON parser with reasoning labels
- **train_sft.py** — Stage 1 supervised fine-tuning
- **train_reward_model.py** — Stage 2A reward model
- **train_ppo.py** — Stage 2A PPO training
- **train_dpo.py** — Stage 2 DPO training
- **eval.py** — evaluate all four models on held-out conversation

- **dev_assistant_finetune.ipynb** — orchestration notebook

Steps:

- Create the folder `My Drive/CSE534_project/` in Google Drive (web or desktop client).
- Upload all eight files to that folder.
- Open the .ipynb file in Colab — right-click in Drive, Open with → Google Colaboratory.
- **Runtime** → **Change runtime type** → **A100** (Pro feature). T4 / L4 also work but slower.
- **Runtime** → **Manage sessions** → **Allow background execution** if you'll close the tab during PPO.

10.3 Colab training cell-by-cell

Run the notebook cells in order. Total wall-clock on A100: ~100 minutes.

#	Cell	What it does	Time
1	GPU check	!nvidia-smi prints the A100 status	instant
2	Mount Drive	drive.mount + cd into project folder	20 s
3	Install deps	pip install unsloth, transformers≥4.56, trl≥0.20, peft≥0.18, bitsandbytes, accelerate, datasets, python-doc	5 min
4	Set persona	PERSONA='dev', JSON filename, HOLDOUT=10	instant
5	Clean checkpoints	rm -rf {persona}-sft, -rm, -dpo, -ppo dirs (avoids stale shard collision)	instant
6	Parse JSON	python parse_json_dataset.py … → sft/dpo/test JSONLs with reasoning traces	10 s
7	Stage 1 SFT	10 epochs, LoRA r=32, lr=2e-4, warmup=0.05	~10 min
8	Stage 2A RM	6 epochs, scalar reward head, lr=5e-6	~5 min
9	Stage 2A PPO	3 passes × 2 epochs, KL=0.15, paged_adamw_8bit	~30 min
10	Stage 2B DPO	12 epochs, beta=0.1, LoRA r=16, attention only	~12 min
11	Eval	all 4 models scored on held-out conversation	~3 min
12	Convert to GGUF	convert_hf_to_gguf.py --outtype q8_0 (~1.6 GB)	~5 min
13	Zip + download	zip {persona}-assistant.gguf for Drive download	~1 min

After cell 11 you'll see the side-by-side comparison table (base / SFT / DPO / PPO). DPO is almost always the best checkpoint to use in production — set **PICK = f'{PERSONA}-dpo-merged'** in cell 12 before exporting. If DPO underperforms (rare), pick whichever model has the highest action F1 with parse rate = 1.0.

10.4 Download the GGUF and register with Ollama

Once cell 13 finishes, the GGUF lives at `My Drive/CSE534_project/dev-assistant.gguf`. Download it to your Mac (Drive web → right-click → Download, or use Drive Desktop sync) into the project folder.

Then register with Ollama using the explicit Qwen2.5 chat template:

```
cd "/Users/amartya/Documents/CSE534 project"

# 1. Verify the file is the new one (timestamp / size sanity check)
ls -la dev-assistant.gguf

# 2. Write the Modelfile. The TEMPLATE block is CRITICAL — without it
# Ollama defaults to {{ .Prompt }} which strips role tags and the
# model falls back to base behavior with hallucinations.
cat > Modelfile <<'EOF'
FROM ./dev-assistant.gguf

TEMPLATE """{{ if .System }}<|im_start|>system
{{ .System }}<|im_end|>
{{ end }}{{ if .Prompt }}<|im_start|>user
{{ .Prompt }}<|im_end|>
{{ end }}<|im_start|>assistant
"""

PARAMETER stop "<|im_end|>"
PARAMETER stop "<|endoftext|>"
PARAMETER temperature 0
EOF

# 3. Remove any older registered version, then create fresh
ollama rm dev-assistant 2>/dev/null
ollama create dev-assistant -f Modelfile

# 4. Verify the template stuck (should print the multi-line block,
# NOT just {{ .Prompt }})
ollama show dev-assistant --template

# 5. Smoke test — should print a 4-field JSON with reason + tool
ollama run dev-assistant "Block tomorrow morning 9 to 11 for the Patel proposal."
```

Expected output of step 5 (formatting may vary slightly):

```
{"reason":"Dev committed to a specific time block for Patel that must be reserved
to prevent conflicts.",
"importance":"High","tool":"create_calendar_event",
"detail":"block 9-11 patel proposal"}
```

10.5 Start the voice UI

Two terminal tabs are needed: one running Ollama, one running the voice UI.

```
# === Terminal 1 — Ollama (keeps the model resident in RAM) ===
ollama serve

# (leave this tab running)
```

```
# === Terminal 2 — Voice UI ===
cd "/Users/amartya/Documents/CSE534 project"
source .venv/bin/activate
PERSONA=dev python voice_ui.py

# Output:
# [boot] persona=dev ollama=dev-assistant
# [boot] log file: ./actions_log.txt
# [boot] loading faster-whisper 'base' (int8 CPU) ...
# [ready] open http://127.0.0.1:5050

# === Optional Terminal 3 — tail the action log live ===
tail -f "/Users/amartya/Documents/CSE534 project/actions_log.txt"
```

Then in any browser:

- Open `http://127.0.0.1:5050`.
- Click the ■ **Start listening** button.
- First time only: macOS will ask for Microphone permission. Allow it.
- Speak naturally. The circle pulses green while listening, turns blue when it hears your voice, amber while thinking. Watch the feed for the model's reasoning + decision and the YES/NO verdict.
- Click ■ **Stop** when done. Closing the browser tab does NOT stop the loop — use the Stop button.

10.6 Tunable knobs (env vars)

Pass these in front of the python voice_ui.py command to adjust runtime behavior without editing code:

Variable	Default	Use when
PERSONA	dev	load a different persona's assistant (arjun/margaret/neel)
OLLAMA_MODEL	{persona}-assistant	the Ollama model is registered under a different name
WHISPER_SIZE	base	tiny = ~3x faster, less accurate; small = slower, more accurate
TTS_VOICE	Samantha	any name from `say -v ?` (e.g. Daniel, Karen)
SILENCE_DBFS	0.006	raise (e.g. 0.012) in noisy rooms; lower (0.003) for quiet voices
TRAIL_SILENCE	1.0	raise (1.5) if it cuts you off when you pause mid-sentence
MAX_UTTERANCE	20.0	cap on a single utterance length, in seconds

Example — quiet voice + a longer pause tolerance:

```
SILENCE_DBFS=0.003 TRAIL_SILENCE=1.5 PERSONA=dev python voice_ui.py
```

10.7 Common troubleshooting

Symptom	Cause	Fix
UI says NO for everything	Ollama still serving the SFT-collapsed model OR the Modelfile lacks the Qwen TEMPLATE	ollama rm + ollama create with the explicit TEMPLATE block in
Model output is prose, not JSON	Modelfile TEMPLATE is the default {{ .Prompt }}	Same fix — add the explicit Qwen TEMPLATE
Model hallucinates dates / training fragments	Same — chat template missing	Same fix
Cuts you off mid-sentence	Trailing-silence threshold too short	TRAIL_SILENCE=1.5 (or 2.0) python voice_ui.py
Doesn't pick up your voice	Energy gate too high	SILENCE_DBFS=0.003 python voice_ui.py
Thinking time >2s	Ollama unloaded the model (idle), or large num_predict	keep_alive=30m is already set; pre-warm with `ollama run dev-
OOM during PPO in Colab	Stale 3B shards mixed with new 1.5B run	Run cell 5 (cleanup) then re-run from cell 6
No microphone permission popup	macOS already denied — no second prompt	System Settings → Privacy & Security → Microphone → enable
Ollama is not running	brew service was stopped	ollama serve (foreground) OR brew services start ollama

11. Limitations and Recommended Improvements

Three limitations are worth highlighting because they constrain how reliably this system performs in production.

11.1 Dataset size

Fewer than 200 labelled utterances per persona is below the threshold at which fine-tuning small language models reliably generalizes. The strongest single intervention to improve accuracy would be to paraphrase each utterance 5–10× using a stronger LLM (Claude or GPT-4o) while keeping the (importance, tool, detail) labels fixed, expanding the training set to ~1500 examples per persona. Class balancing (oversampling Alarm and Note examples to match Reminder counts) would also help.

11.2 Mode collapse on SFT

SFT alone is unreliable on this dataset because the null-tool class is the majority. DPO is the safe production choice for this reason. If SFT must be retained as a baseline, the loss should be class-weighted to discourage majority-class predictions.

11.3 Reward model quality

The reward model is itself trained on small data (~165 preference pairs). PPO is therefore optimizing against a noisy reward signal, and any reward hacking the policy discovers will be rewarded. A larger and more diverse preference dataset — ideally with human-rated rather than synthetic preferences — would significantly improve PPO stability. Alternatively, KTO (Kahneman-Tversky Optimization) is a one-sided variant of DPO that requires only thumbs-up / thumbs-down labels rather than paired preferences and would be easier to scale with human feedback.

11.4 Action layer

The current rule-based YES/NO layer is fast but loses information — a YES doesn't actually create the calendar event, just records the intent. Reintroducing the AppleScript MCP layer for confirmed YES decisions, asynchronously after the voice response, would restore the original action-taking behaviour without paying the latency cost on the user-perceived path. The MCP server (mac_actions_mcp_server.py) is already written; only the integration glue needs to be added.

12. Source Files

File	Purpose
parse_json_dataset.py	Persona JSON → SFT, DPO, test JSONL
train_sft.py	Stage 1 supervised fine-tuning
train_reward_model.py	Stage 2A reward model (RLHF)
train_ppo.py	Stage 2B PPO training (RLHF)
train_dpo.py	Stage 2 DPO training (alternative branch)
eval.py	Score all four models on held-out test
dev_assistant_finetune.ipynb	Colab orchestration notebook
voice_ui.py	Web UI + listening loop + rule layer
mac_actions_mcp_server.py	AppleScript-backed MCP server (optional)
actions_log.txt	Append-only log of YES/NO decisions